

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-213БВ-25

Студент: Дадаханов Н. С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 01.10.26

Москва, 2026

Постановка задачи

Вариант 5.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы

Пользователь вводит команды вида: «число». Далее это число передается от родительского процесса в дочерний. Дочерний процесс производит проверку на простоту. Если число составное, то в это число записывается в файл. Если число отрицательное или простое, то тогда дочерний и родительский процессы завершаются.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void);` - создает дочерний процесс.
- `int pipe(int pipefd[2]);` - создает неименованный канал для передачи данных между процессами.
- `int dup2(int oldfd, int newfd);` - перенаправляет файловый дескриптор.
- `int execl(const char *path, const char *arg0, ...);` - заменяет текущий образ процесса исполняемой программой.
- `int open(const char *pathname, int flags, mode_t mode);` - открывает или создает файл.
- `int close(int fd);` - закрывает файловый дескриптор.
- `ssize_t read(int fd, void *buf, size_t count);` - считывает байты из дескриптора в буфер.
- `ssize_t write(int fd, const void *buf, size_t count);` - записывает байты из буфера в дескриптор.
- `pid_t wait(int *wstatus);` - ожидает завершения дочернего процесса.
- `void _exit(int status);` - завершает выполнение процесса.

В рамках лабораторной работы были написаны две отдельные программы на языке Си (`parent.c` и `child.c`), а также файл сборки `Makefile`.

Как работает программа:

1. Запуск и ввод файла:

родительский процесс запускается, просит пользователя ввести имя файла и сохраняет его.

2. Создание каналов:

родитель создает два пайпа с помощью `pipe`: `pipe1` (для отправки чисел ребенку) и `pipe2` (для получения ответа от ребенка).

3. Создание дочернего процесса:

- С помощью `fork()` процесс делится на родителя и потомка.
- В дочернем процессе через `dup2()` стандартный ввод (`STDIN`) заменяется на `pipe1`, а стандартный вывод (`STDOUT`) — на `pipe2`.
- Дочерний процесс закрывает ненужные копии дескрипторов и вызывает `execl()`, запуская отдельную программу `./child` и передавая ей имя файла через аргументы.

4. Обмен данными:

- Родитель в цикле считывает с клавиатуры числа и пишет их в `pipe1`.
- Дочерний процесс читает число из своего ввода и проверяет его на простоту:
 - Если число **составное** — ребенок записывает его в открытый файл и отправляет родителю в `pipe2` символ `'C'` (продолжить работу).
 - Если число **простое или** ≤ 1 (включая отрицательные) — ребенок отправляет родителю символ `'T'` (завершить работу) и выходит.

5. Завершение работы:

получив от ребенка сигнал `'T'`, родитель выходит из цикла, закрывает оставшиеся концы каналов и через `wait()` дожидается завершения дочернего процесса.

Код программы

child.c

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
int read_line(int fd, char* buf, int max_len) {  
    int i = 0;  
    char c;  
    int res;  
    while (i < max_len - 1 && (res = read(fd, &c, 1)) > 0) {  
        if (c == '\n') break;  
        buf[i++] = c;  
    }  
    buf[i] = '\0';  
    if (res <= 0 && i == 0) return -1;  
    return i;  
}
```

```
int str_to_int(const char* s) {  
    int res = 0;  
    int sign = 1;  
    int i = 0;  
    if (s[0] == '-') {  
        sign = -1;  
        i++;  
    }  
    for (; s[i] >= '0' && s[i] <= '9'; i++) {  
        res = res * 10 + (s[i] - '0');  
    }  
    return res * sign;  
}
```

```
int is_prime(int n) {  
    if (n < 2) return 0;  
    for (int i = 2; i * i <= n; i++) {  
        if (n % i == 0) return 0;  
    }  
    return 1;  
}
```

```
int main(int argc, char* argv[]) {  
    if (argc < 2) return 1;
```

```
    int file_fd = open(argv[1], O_WRONLY | O_CREAT | O_APPEND, 0644);
```

```

if (file_fd < 0) {
    write(STDOUT_FILENO, "T", 1);
    return 1;
}

char buf[128];
int len;

while ((len = read_line(STDIN_FILENO, buf, sizeof(buf))) != -1) {
    if (len == 0) continue;

    int num = str_to_int(buf);

    if (num <= 1 || is_prime(num)) {
        write(STDOUT_FILENO, "T", 1);
        break;
    }

    write(file_fd, buf, len);
    write(file_fd, "\n", 1);

    write(STDOUT_FILENO, "C", 1);
}

close(file_fd);
return 0;
}

```

parent.c

```

#include <unistd.h>
#include <sys/wait.h>

int read_line(int fd, char* buf, int max_len) {
    int i = 0;
    char c;
    int res;
    while (i < max_len - 1 && (res = read(fd, &c, 1)) > 0) {
        if (c == '\n') break;
        buf[i++] = c;
    }
    buf[i] = '\0';
    if (res <= 0 && i == 0) return -1;
    return i;
}

void print(const char* s) {

```

```

int len = 0;
while (s[len] != '\0') len++;
write(STDOUT_FILENO, s, len);
}

int main(void) {
    char filename[128];

    print("Enter filename: ");
    int file_len = read_line(STDIN_FILENO, filename, sizeof(filename));
    if (file_len <= 0) {
        return 1;
    }

    int pipe1[2];
    int pipe2[2];
    pipe(pipe1);
    pipe(pipe2);

    pid_t pid = fork();

    if (pid == 0) {
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execl("./child", "./child", filename, (char*)0);
        _exit(1);
    }

    close(pipe1[0]);
    close(pipe2[1]);

    char buf[128];
    print("Enter numbers:\n");

    while (1) {
        print("> ");
        int len = read_line(STDIN_FILENO, buf, sizeof(buf));
        if (len == -1) {
            break;
        }
    }
}

```

```

write(pipe1[1], buf, len);
write(pipe1[1], "\n", 1);

char status;
if (read(pipe2[0], &status, 1) <= 0) {
    break;
}

if (status == 'T') {
    print("Child terminated (prime or <= 1). Exiting.\n");
    break;
}
else if (status == 'C') {
    print("Composite number recorded in file.\n");
}
}

close(pipe1[1]);
close(pipe2[0]);

wait((int*)0);
return 0;
}

```

Makefile

CC = gcc

CFLAGS = -Wall -Wextra -std=c11 -pedantic

all: parent child

parent: parent.c

\$(CC) \$(CFLAGS) parent.c -o parent

child: child.c

\$(CC) \$(CFLAGS) child.c -o child

clean:

rm -f parent child

Протокол работы программы

```
nazivin2007@LAPTOP-FEJQE646:~/OS_lab01$ make
gcc -Wall -Wextra -std=c11 -pedantic parent.c -o parent
gcc -Wall -Wextra -std=c11 -pedantic child.c -o child
nazivin2007@LAPTOP-FEJQE646:~/OS_lab01$ ./parent
Enter filename: output.txt
Enter numbers:
> 4
Composite number recorded in file.
> 8
Composite number recorded in file.
> 15
Composite number recorded in file.
> 9
Composite number recorded in file.
> 7
Child terminated (prime or <= 1). Exiting.
nazivin2007@LAPTOP-FEJQE646:~/OS_lab01$ cat output.txt
4
8
15
9
nazivin2007@LAPTOP-FEJQE646:~/OS_lab01$ ./parent
Enter filename: negative_test.txt
Enter numbers:
> 12
Composite number recorded in file.
> -6
Child terminated (prime or <= 1). Exiting.
nazivin2007@LAPTOP-FEJQE646:~/OS_lab01$ cat negative_test.txt
12
nazivin2007@LAPTOP-FEJQE646:~/OS_lab01$
```

Вывод

В ходе выполнения лабораторной работы были освоены механизмы управления процессами в операционной системе Linux и организация межпроцессного взаимодействия (IPC) через системные неименованные каналы (**pipe**). Были практически отработаны системные вызовы **fork**, **execl**, **dup2**, **pipe**, **wait**, а также организация низкоуровневого файлового ввода-вывода без применения стандартной библиотеки **<stdio.h>**.